From the code, we will know that Devise will attach some default modules such as Database Authenticable, Registerable, Recoverable, Rememberable, Trackable, and Validatable. As I wrote earlier in this chapter, I suppose you already knew what the modules are for.

At this point, you have prepared all the basic settings that a Rails application needs to implement Devise. So, the next step is creating the table on your database by migrating the migration file. If you don't make any change to the Devise migration file, it means Devise will only generate columns for its default modules. But, if you make some changes like commenting on other modules such as `t.encryptable`, `t.confirmable`, `t.lockable`, and `t.tocken_authenticatble`, you will have extra columns in your user's table that will handle some specific Devise modules. So, it depends on your requirement whether you are going to use the modules or not.

We have prepared our migration file, now let's create the table. I presume that you already have the database and have prepared the database configuration at `config/database.yml`. If so, all you need to do is execute this command:

```
$ rake db:migrate
```

Now, you have prepared everything to make Devise run smoothly on your Rails application. But, there's one more thing that I want to show you. It's about how to wrap controllers with your authorization and see it in action.

Run your first application with Devise

In this section, we are going to talk about how to wrap your controllers with Devise authorization and use some Devise helper in your views. First, I want to generate a single controller by executing this command:

```
$ rails generate controller home index
```

This command will generate the controller (`home_controller.rb`) with an action named `index`. It also generates a view file located at `views/home/index.html.erb`. Let's start by opening the controller file and add a code (`:authenticate_user!`) between class definition and first action definition. Why `:authenticate_user!`? As I stated before, we have our Devise model named as user and this code is one of the Devise helpers that I meant. So, in the future, when you have a Devise model with a different name, you can change the user part in the code with your actual model name. According to our example, the controller code will be like the following:

```
class HomeController < ApplicationController
before_filter :authenticate_user!
```


www.it-ebooks.info